TURBINE

(Disclaimer: this document is a very heavy wip)

(Also some things look bad right now because this used to be a .txt)

OVERVIEW
Turbine is a 16-bit CPU with a CISC architecture.

Turbine uses a Von Neumann architecture and sees memory as 64K 16-bit words.
This CPU can be seen as a simple yet very powerful architecture with some extra features Sprinkled on top. The “normal” instructions are all fully orthogonal, meaning any combination of Condition, source, operation and destination is possible.

This document was written assuming prior knowledge on CPUs and every notation of binary is done with the most significant bit on the left.

REGISTERS

Turbine has both “normal” registers and “special” registers. This is an overview of the normal registers, the special ones will be covered later on in this document.

A: general-purpose, but hard-wired to the ALU's second input

B, C, D, E, F: general-purpose

PC: program counter, reading from it returns PC's value when the instruction was first ran, writing to it will instantly update the address output.

flags:

zero (Z): set by most instructions when the data passing through the ALU is zero ($0000)

carry (C): set whenever the ALU produces a carry-out of sorts, such as addition overflows or shifting a bit out of a word.

Interrupt: automatically reset whenever an interrupt occurs and set when the return from interrupt instruction is run, but can also be manually set or reset. The CPU can only respond to interrupts when this flag is set, but will respond to non-maskable interrupts regardless of the status of this flag.

INSTRUCTION FORMAT: CCC OOOOO SSSS DDDD

C: condition (COND)
O: operation (OPER)
S: source (SRC)
D: destination (DST)

CONDITION
All of Turbine's instructions have a condition that determines whether or not the instruction gets executed. The condition operates on the states of the zero and carry flags. If an instruction's condition fails, the instruction, along with any immediates it has, get skipped over.

000: Z
001: !Z
010: C
011: !C
100: Z AND C
101: Z OR C
110 Z XOR C
111: always

OPERATION
This operand controls what operation the ALU will perform. Operations are always done between the source (SRC) and the A register.

00000: MOV
moves the source to the destination.
flags affected: Z

00001: iMOV
a MOV instruction that doesn't affect any flags

00010: ADD
adds SRC and A
flags affected: Z,C

00011: iADD
an ADD instruction that doesn't affect any flags

00100: ADC
adds SRC and A with carry in
flags affected: Z,C

00101: SUB
subtracts SRC and A
flags affected: Z,C
Assembler:

00110: iSUB
a SUB instruction that doesn't affect any flags

00111: SBC
subtracts SRC and A with carry in
flags affected: Z,C

01000: INC
increments SRC
flags affected: Z,C

01001: iINC
an INC instruction that doesn't affect any flags

01010: DEC
decrements SRC
flags affected: Z,C

01011: iDEC
a DEC instruction that doesn't affect any flags

01100: NEG
negates SRC (2's compliment, so invert then increment)
flags affected: Z

01101: iNEG
a NEG instruction that doesn't affect any flags

01110: NOT
bitwise NOT on SRC
flags affected: Z

01111: AND
bitwise AND between SRC and A
flags affected: Z

10000: NAND
bitwise NAND between SRC and A
flags affected: Z

10001: OR
bitwise OR between SRC and A
flags affected: Z

10010: NOR
bitwise NOR between SRC and A
flags affected: Z

10011: XOR
bitwise XOR between SRC and A
flags affected: Z

10100: XNOR
bitwise XNOR between SRC and A
flags affected: Z

10101: SHL
logical shift left on SRC
0>SRC[15,0]>C
flags affected: Z,C

10110: iSHL
a SHL instruction that doesn't affect any flags

10111: SLC
logical shift left on SRC with carry in
C>SRC[15,0]>C
flags affected: Z,C

11000: SHR
logical shift right on SRC
C<SRC[15,0]<0
flags affected: Z,C

11001: SRC
logical shift right on SRC with carry in
C<SRC[15,0]<C
flags affected: Z,C

11010: ASR
arithmetic shift right on SRC
(this instruction may be replaced in the future but im not sure yet)
0>SRC[14,0]>C
SRC[15] is unaffected
flags affected: Z,C

11011: iASR
an ASR instruction that doesn't affect any flags

11100: ROL
rotate SRC to the left one bit
C<SRC[15,0]<SRC[15]
flags affected: Z,C

11101: ROR
rotate SRC to the right one bit
SRC[0]>SRC[15,0]>C
flags affected: Z,C

11110: MIR
mirrors SRC by reversing the order of it's bits
SRC[15,0] -> SRC[0,15]
flags affected: Z

11111: SWA
swaps the two bytes within SRC around
(SRC[15,8],SRC[7,0]) -> (SRC[7,0],SRC[15,8])
flags affected: Z

sources/destinations:
0000: A
0001: B
0010: C
0011: D
0100: E
0101: F
0110: PC
0111: immediate
1000: memory at A
1001: memory at B
1010: memory at C
1011: memory at D
1100: memory at E
1101: memory at F
1110: primary stack
1111: memory at immediate (absolute)

addressing modes:
register: the source or destination is the specified register.
Assembler: R

immediate: the source is provided by the word following the instruction. An instruction's destination cannot be an immediate, unless the instruction is affected by NDXD (page idk).
Assembler: 16-bit value, prefixes: none for decimal, $ for hex, % for binary

memory at register: the specified register is used as a memory pointer, and the word in memory pointed at is used as the source or destination.
Assembler: (R)

stack: pushes/pulls from the main stack.
Assembler: S1 or PS

absolute: the word following the instruction is used as a memory pointer, and the word in memory pointed at is used as the source or destination.
Assembler: same as immediate but in parenthesis

useful macros:
111 00001 0000 0000: NOP
CCC 00001 0110 0110: halt
CCC 00001 0111 0110: return from subroutine

SPECIAL INSTRUCTIONS

These instructions are used to access special registers, run useful instructions and perform basic and complex operations. They are accessed by setting the destination of a normal instruction to an immediate and occasionally use a different instruction format.

PRIMARY STACK POINTER

The primary stack pointer (SP) register determines where in memory a primary stack push or pull will read data from or write data to. The stack cascades downwards by default, but it can be set to cascade upwards by setting a bit in the status register (see “status register” page idk). Upon reset, it is initialized to $7FFF. The stack pointer gets post-decremented when pushing to the stack and pre-incremented when pulling from the stack. If the stack pointer is set to $7F00 and is set to cascade downwards, a pull will read the data at memory address $7F00 and increment the stack pointer, and a push will write data to memory address $7EFF and decrement the stack pointer.

Stack pointer write: CCC 00000 SSSS 0111
Assembler: MOV SRC,SP
Stack pointer read: CCC 00001 DDDD 0111 (DST cannot be an immediate)
Assembler: MOV SP,DST
Both affect the Z flag

I/O PORT

Turbine has one native I/O port at vector $0000 (address $0000 and VPB pin low). It is intended for quick-access simple I/O applications.

I/O write: CCC 00010 SSSS 0111
Assembler: MOV SRC > IO
I/O read: CCC 00011 DDDD 0111 (DST cannot be an immediate)
Assembler: MOV IO > DST
Both affect the Z flag

AUXILIARY STACK

This is a stack with 16 layers contained within the CPU. The pointer for this stack is found within the status register. The auxiliary stack pointer cascades downwards and points to the last stack entry. The auxiliary stack pointer is set to $F upon reset, but the contents of the stack do not get initialized.

Auxiliary stack push: CCC 00100 SSSS 0111
Assembler: MOV SRC > S2
Auxiliary stack pull: CCC 00101 DDDD 0111 (DST cannot be an immediate)
Assembler: MOV S2 > DST
Both affect the Z flag

STATUS REGISTER

The status register provides information on what the CPU is doing. It is formatted as follows:

bit 15: toggled when the swap state instruction is ran (read only)
bit 14: set while an NMI is currently taking place (read only)
bit 13: set while an interrupt is currently taking place (read only)
bit 12: set while an interrupt is queued (read only)
bit 11: interrupt flag
bit 10: primary stack pointer direction
bit 9: carry flag
bit 8: zero flag
bits 7-4: unused but writeable
bits 3-0: auxiliary stack pointer

Status register write: CCC 00110 SSSS 0111
Assembler: MOV SRC > STAT
Status register read: CCC 00111 DDDD 0111 (DST cannot be an immediate)
Assembler: MOV STAT > DST
Both affect the Z flag

FLAG INSTRUCTIONS
Set zero flag: CCC 00001 0111 0111
Assembler: SETZ
Clear zero flag: CCC 00011 0111 0111
Assembler: CLRZ
Set carry flag: CCC 00101 0111 0111
Assembler: SETC
Clear carry flag: CCC 00111 0111 0111
Assembler: CLRC
Invert zero flag: CCC 01001 0111 0111
Assembler: INVZ
Invert carry flag: CCC 01011 0111 0111
Assembler: INVC
Set interrupt flag: CCC 01101 0111 0111
Assembler: SETI
Clear interrupt flag: CCC 01111 0111 0111
Assembler: CLRI

Jump to subroutine: CCC 01000 SSSS 0111
This instruction pushes a return address onto the primary stack and jumps to SRC.
Assembler: JSR SRC

Return from interrupt: CCC 01001 0000 0111
If there is no interrupt queued, this instruction pulls a return address from the primary stack and jumps to it and the interrupt flag is set. If an interrupt is queued, this instruction jumps to where the interrupt vector points to.
Assembler: RETI

Compare: CCC 01010 SSSS 0111
This instruction subtracts A from SRC, but doesn't do anything with the difference. The zero and carry flags are set accordingly.
Assembler: CMP SRC,A

STATE EXCHANGING

Turbine possesses the ability to internally save its current state and load a saved state. Here is how each register gets saved:
A, B, C, D, E, F, SP registers: saved as-is
PC: PC+1 is saved
Status register: only bits 11 through 4 are saved. Loading a state does not affect the unsaved bits.

Save state: CCC 01011 0000 0111
Assembler: SVST
Load state: CCC 01101 0000 0111
Assembler: LDST
Swap state: CCC 01111 0000 0111
This powerful instruction also toggles bit 15 of the status register. It can be used for primitive multithreading.
Assembler: SWST

INDEX SOURCE/DESTINATION INSTRUCTIONS

Turbine has 2 instructions that enable it to do indexed memory accesses: iNDeX Source (NDXS) and iNDeX Destination (NDXD). These instructions (henceforth grouped together as “NDX”) modify either the source or destination of the instruction they precede by treating it as a base address onto which an offset, provided by the NDX instruction, is added. Instructions modified by an NDX instruction take one extra cycle to complete, during which they access the indexed memory address. NDX instructions also come with some rules of their own:
-If either of the conditions of an NDX instruction or the instruction it modifies fail, then neither will be executed.
-If an interrupt (maskable or non-maskable) is queued while an NDX instruction is running, it will remain queued until the modified instruction is done being executed.
-NDXD cannot be applied to a special instruction, it will instead be treated as a normal instruction that uses an immediate as the base address. However
-NDXS can be applied to a special instruction that uses it's source operand as a destination to have the destination be indexed.
-If PC (0110) is used as a source or destination that gets modified by an NDX instruction, then the last entry on the auxiliary stack is pulled and used as the base address instead of PC.

Index source: CCC 01100 SSSS 0111
Flags affected: none
Index destination: CCC 01110 SSSS 0111
Flags affected: none

Bit check: CCC 1BBBB SSSS 0111
This instruction sets the carry flag to one of the bits within SRC. The bit checked is determined by the B operand.

Assembler: BCHK SRC,BIT
BIT = value between 0 and 15 in hex or decimal (no prefix needed)

INTERRUPTS
Turbine has two interrupt inputs, one for normal interrupts and one for non-maskable interrupts. Both inputs are sampled when the clock is low and are triggered when the current instruction has finished executing. Furthermore, normal interrupts can only be triggered if the interrupt flag in the status register is set. When a normal interrupt is triggered, the CPU pushes a return address to the primary stack and jumps to where the interrupt vector at memory address $FFFD points. Non-maskable interrupts (NMIs) take priority over normal interrupts and can be triggered at any given time, except when the CPU is already handling an NMI. The NMI vector is located at memory address $FFFE.

RESETS
A reset will initialize every register within the CPU (minus the contents of the auxiliary stack) to their default values. Like the interrupt inputs, the reset input is sampled when the clock is low, but a reset will occur immediately instead of waiting for the current instruction to finish. A reset only takes one clock cycle, during which it fetches the contents of the reset vector at memory address $FFFF and jumps to it. PC's saved state is also set to the the contents of the reset vector upon reset.

I hope you find the joy in using this CPU that I did not find in making it.